

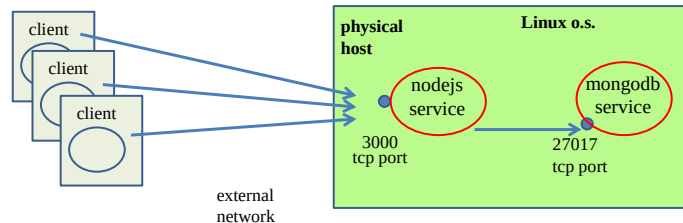
Esempio di applicazione con nodejs e mongodb realizzata mediante docker-compose (ASWh4)

Prof. Vittorio Ghini - 3/04/2025

L'esempio di applicazione da realizzare.

Vediamo passo per passo come costruire una applicazione web, implementata mediante **nodejs**, che sfrutta le librerie **express** per interfacciarsi un database gestito dal dbms **mongodb**.

Struttura dell'applicazione



Cosa fa l'applicazione:

Il server web nodejs accetta richieste http:

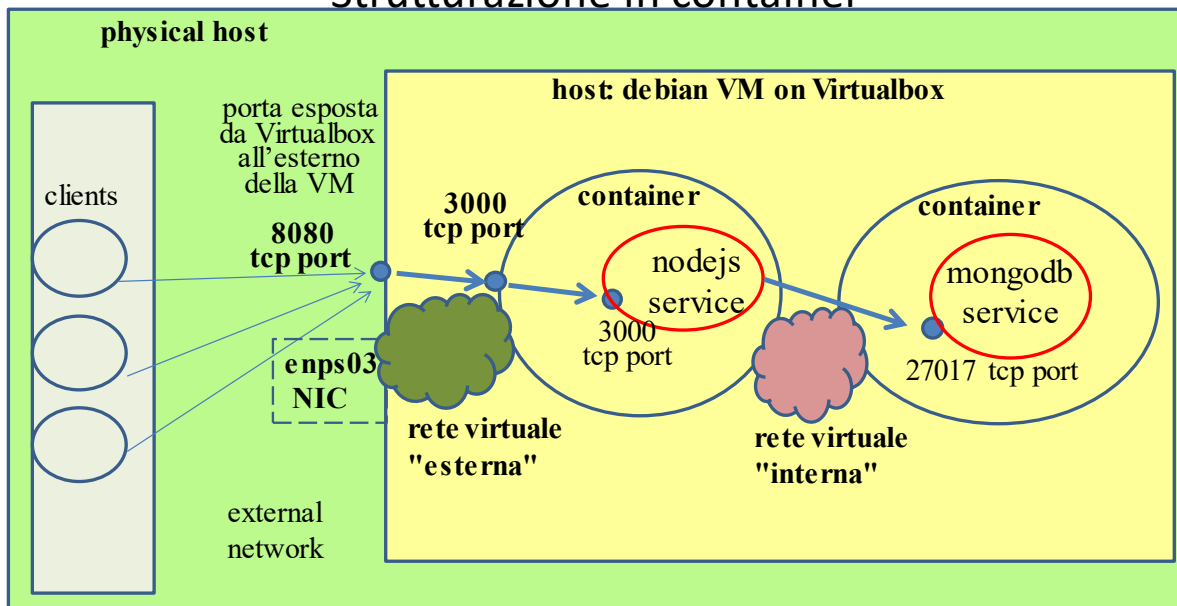
- di tipo **POST** per il percorso **/submit** contenenti due parametri seq1 e seq2, poi calcola due nuove stringhe, funzione dei due parametri, salva nel database la quadrupla formata dalle due stringhe vecchie e dalle due calcolate, infine restituisce al client la quadrupla.
- di tipo **GET** per il percorso **/show** e restituisce al client il contenuto del database.
- di tipo **GET** per il percorso **/** che restituisce al client una pagina HTML con una form in cui inserire due stringhe seq1 e seq2. Un pulsante "Submit" manda queste due stringhe al web server mediante un POST sulla pagina /submit affinché vengano inserite nel database. La richiesta alla POST (la quadrupla) viene visualizzata dal browser.

Struttura in container dell'applicazione:

Ciascuno dei due componenti, uno nodejs con express e l'altro mongodb, viene realizzato e pacchettizzato in un proprio **container docker**.

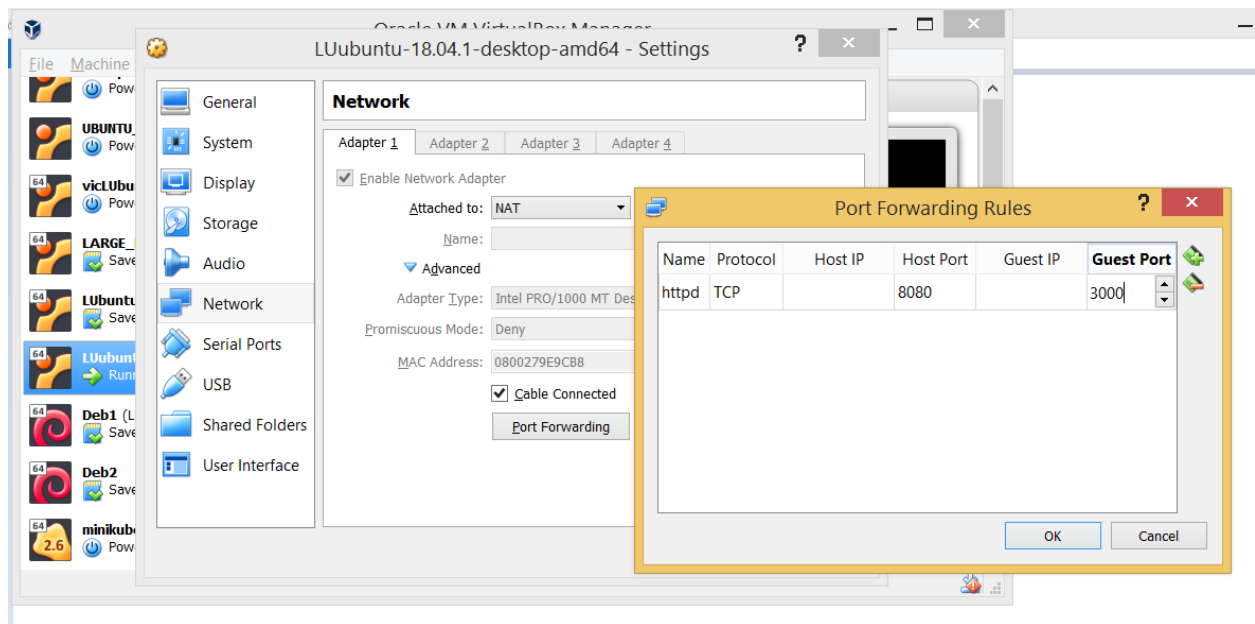
L'applicazione nel suo complesso (formata dai due container) viene costruita ed eseguita in una macchina Linux, sfruttando i pacchetti docker e docker-compose.

Strutturazione in container



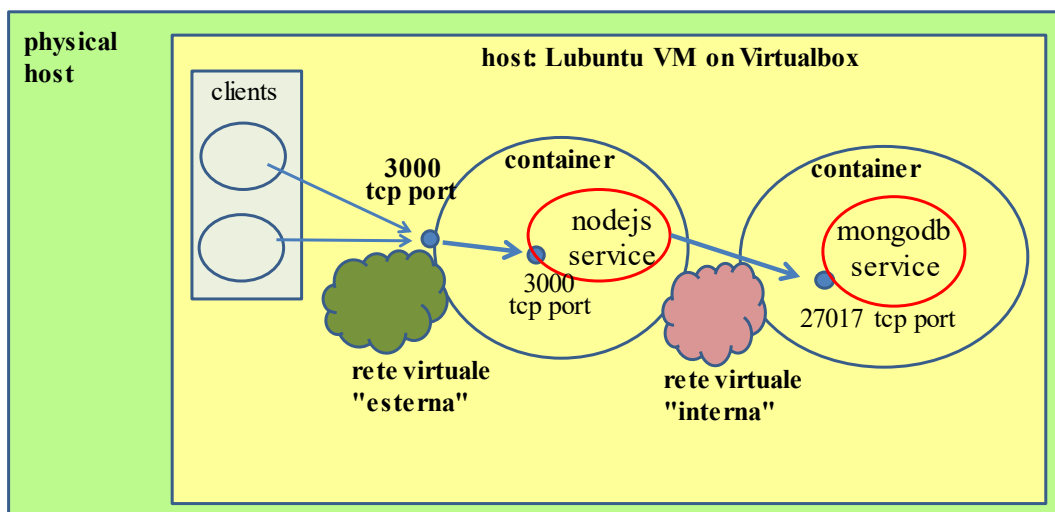
La struttura dell'applicazione, e la sua organizzazione in container è rappresentata nello schema seguente, in cui è indicato il caso in cui l'applicazione è eseguita su un o.s. Linux ospitato all'interno di una macchina virtuale (gialla) gestita dall'hypervisor virtualbox; nello schema compaiono anche i client web (browser), che non consideriamo parte dell'applicazione. Ovviamente, nulla vieta che l'applicazione venga eseguita direttamente su una macchina fisica con o.s. Linux.

Qualora l'applicazione venga eseguita all'interno di una macchina virtuale in Virtualbox, occorrerà configurare virtualbox affinché renda visibile all'esterno della macchina virtuale la porta su cui si vuole ricevere le connessioni provenienti dai client. Si veda a tale proposito la figura che descrive come configurare il port forwarding in virtualbox, esponendo la porta 3000, pubblicata dal container, sulla porta esterna 8080 della VM.



Nulla vieta che i test vengano eseguiti collocando i clients all'interno della macchina virtuale stessa, come da schema nella seguente figura.

Strutturazione in container test con client collocati nella macchina virtuale



Solo in tal caso, non occorrerà più configurare Virtualbox per esporre la porta del servizio fuori dalla macchina virtuale stessa; i clients si collegheranno alla porta esposta esternamente al container di nodejs.

Nota bene: il codice sorgente dell'applicazione e tutti i file che servono a creare i container dell'applicazione e a metterli in esecuzione (in pratica tutti i file citati in questo documento) sono contenuti nell'archivio zippato **ASWh4.zip**

Organizzazione del documento

Questo documento è organizzato in 10 parti:

1) Installazione di docker compose.

2) **Costruzione della rete virtuale "interna" che connette tra loro i due container del web server nodejs e di mongodb e costruzione della rete virtuale "esterna" che fa comunicare il container del web server con la VM host e con eventuali container con ruolo di gateway.**

3) **Costruzione ed esecuzione del container con dentro l'applicazione web basata su nodejs** ed express, con particolare accento sul modo di garantire che il servizio nodejs riesca collegarsi al servizio del container mongodb. Ciò sarà garantito collocando entrambi i container in una rete virtuale, precedentemente create, che chiameremo "interna" che automaticamente realizza un DNS implicito di docker. In tal modo, il nome del container di mongodb potrà essere usato come nome dell'host virtuale in cui mongodb è in esecuzione e potrà quindi essere risolto in un indirizzo IP mediante il quale nodejs si collega a mongodb.

5) **Costruzione dell'immagine del container con mongodb ed il database.** Vedremo come realizzare il container con il dbms mongodb che mantiene e gestisce il database dbsa con una collection 'alignments'. Definiremo un file Dockerfile che consente di creare (build) l'immagine del container con mongodb ed il nostro database.

6) **Esecuzione del container con mongodb ed il database dbsa.** Indicheremo poi come effettuare manualmente il dispiegamento e l'esecuzione del container ed il controllo del log del container in caso di errori.

7) **Esecuzione del container con dentro l'applicazione web** basata su nodejs, express e mongoose ed il controllo del log del container in caso di errori.

8) **Automatizzazione di costruzione, dispiegamento ed esecuzione di tutti i servizi e dei loro container.**

Armonizzazione ed Automatizzazione di costruzione, dispiegamento ed esecuzione di tutti i container necessari all'applicazione e dell'ambiente di esecuzione dei container, ed infine terminazione dell'applicazione, mediante **docker-compose**. Controllo del log dei servizi in caso di errori mediante docker-compose.

1) Installazione del pacchetto docker-compose in una macchina Linux.

1.1) Installazione di docker in una macchina ubuntu mediante script di convenienza.

```
# per installare docker occorre essere tra i sudoers

# installare curl
sudo apt-get update
sudo apt-get install curl

# installare docker engine
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# verifica che docker sia installato
sudo docker -v
```

1.2) Come eseguire il comando docker senza dover usare sudo.

```
# inserire utente corrente nel gruppo docker
sudo usermod -aG docker ${USER}
# verificare che utente corrente è nel gruppo docker
id -nG
# ricaricare utente
su - ${USER}

# verifica che docker sia eseguibile senza sudo
sudo docker -v
```

1.3) Installazione di docker-compose in una macchina debian in cui è già stato installato docker.

Supponendo di avere già installato docker configurando il repository di docker, non quello di ubuntu, dovrebbe essere sufficiente eseguire i due seguenti comandi:

- sudo apt update
- sudo apt install docker-compose

2) Costruzione della rete virtuale "interna" che connette tra loro i due container del web server nodejs e di mongodb e costruzione della rete virtuale "esterna" che fa comunicare il container del web server con la VM host.

Creo una rete virtuale che verrà usata per comunicare tra i container e la chiamo "interna". Lo scopo vero è non dover definire degli indirizzi IP per i container e sfruttare invece i nomi dei container come se fossero dei nomi di host, che verranno risolti in indirizzi IP

dal DNS implicito che docker realizza automaticamente per ciascuna rete virtuale che crea. I container che usano una stessa rete virtuale possono vedere gli altri container di quella rete usando i nomi dei container come se fossero dei nomi di host, ed il DNS implicito li risolve. Creo la rete virtuale usando il comando:

```
docker network create -d bridge --internal interna
```

Verificare l'esistenza della rete con il comando

```
docker network ls
```

E, per vedere più dettagli sulla rete denominata "interna", eseguire:

```
docker network inspect interna
```

Creo la rete virtuale "esterna" che connette il web server all'host, usando il comando:

```
docker network create -d bridge esterna
```

3) Costruzione dell'immagine del container con dentro l'applicazione web basata su nodejs, express e mongoose

Il codice contiene una directory ASWh4 e due sottodirectory nodejs e mongodb in cui ci sono i file per costruire il container rispettivamente per l'applicazione web e per il database documentale.

Andare nella directory ./ASWh4/nodejs in cui sono presenti i file necessari.

```
./app
./app/package-lock.json
./app/package.json
./app/index.js
./app/src
./app/src/controllers
./app/src/controllers/controller.js
./app/src/lib
./app/src/lib/sequences_alignment.js
./app/src/routes
./app/src/routes/routes.js
./app/src/models
./app/src/models/alignmentModels.js
./Dockerfile
```

C'è un **Dockerfile** ed una sottodirectory app che contiene tre files index.js package.json e package-lock.json ed una sottodirectory src a sua volta contenente 4 sottodirectory: controllers,lib,models,routes. Le quattro sottodirectory contengono l'applicazione vera e propria.

Il file package.json contiene le dipendenze dell'applicazione, cioè le informazioni che servono a capire quali pacchetti per nodejs installare per poter eseguire correttamente tutte le funzioni richiamate nel codice javascript dell'applicazione.

Il file index.js è quello che deve essere eseguito per far partire l'applicazione vera e propria, mediante il comando:

```
node index.js
```

All'interno del file index.js **c'è la parte che stabilisce la connessione tra il web service ed il dbms mongodb** mediante la libreria express. Il file index.js è così fatto:

```
// file index.js

var express = require('express');
var bodyParser = require('body-parser');
var mongoose = require('mongoose')
var Alignment = require('./src/models/alignmentModels')

//Creo istanza di express (web server)
var app = express();

//importo parser per leggere i parametri passati in POST
```

```

app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

//connessione al db
// il secondo mongodb, quello colorato in rosso, e' il nome del
// container di mongo o, usando docker-compose, il nome del servizio
// mongodb specificato nel compose file, che vedremo essere mongodb.

mongoose.connect('mongodb://mongodb/dbsa', { useNewUrlParser: true,
useFindAndModify: false });

//mongoose.connect('mongodb://username:password@host:port', {
//  useNewUrlParser: true, useFindAndModify: false });

var routes = require('./src/routes/routes');
routes(app);

//metto in ascolto il web server
app.listen(3000, function () {
  console.log('Node API server started on port 3000!');
});

```

Notare che l'istruzione:

```

mongoose.connect('mongodb://mongodb/dbsa', { useNewUrlParser: true,
useFindAndModify: false });

```

usa il nome del container (**mongodb, quello colorato in rosso**) come se fosse un nome di host, poiché sappiamo che il container di mongodb lo collocheremo nella stessa rete virtuale chiamata "interna" del container dell'applicazione, e quindi il DNS implicito riuscirà a risolvere il nome del container nell'indirizzo IP del container all'interno della rete virtuale "interna".

Notare anche l'istruzione generica commentata

```

//mongoose.connect('mongodb://username:password@host:port', {
//  useNewUrlParser: true, useFindAndModify: false });

```

che ricorda come sia possibile inserire anche uno username ed una password per accedere a mongodb. Nel nostro caso non è necessario poiché abbiamo configurato mongodb per non richiedere username e password.

Poi scriviamo (c'è già) il **Dockerfile** nella directory nodejs per costruire l'immagine del container nodejsapp dell'applicazione web.

```

FROM ubuntu:xenial

ENV WORKINGDIR=/root/app
# indica dove verranno eseguiti i comandi indicati da RUN
WORKDIR ${WORKINGDIR}

RUN mkdir -p ${WORKINGDIR} && chmod 666 ${WORKINGDIR}

# copia tutta la directory app e le sottodirectory nel container
COPY ./app ${WORKINGDIR}/

```



```

# installo nel container i pacchetti necessari
# e poi butto gli archivi non più utili
RUN apt-get -y update                && \
      apt-get -y install apt-utils    && \
      apt-get -y install nodejs       && \
      apt-get -y install npm          && \
      apt-get -y clean
# installo nel container le dipendenze indicate in package.json
RUN npm install

# ricordo che devo rendere accessibile dall'esterno del container
# la porta 3000 su cui nodejs rimane in attesa di connessioni
# da parte dei client
# Occhio, non espongo la porta, questa EXPOSE è solo una nota per
# ricordare che devo esporre la porta quando eseguo il container.
EXPOSE 3000

# dico quale comando eseguire automaticamente quando eseguirò il
# container: serve per far partire il servizio web nel container.
CMD nodejs index.js

```

A questo punto è tutto pronto, posso costruire l'immagine del container con nodejs e la nostra applicazione, andando nella directory nodejs contenente il Dockerfile, ed eseguendo il comando:

```
docker build -t nodejsapp .
```

Il comando impiega un poco di tempo e poi finisce scrivendo su disco l'immagine di un container con nome nodejsapp.

Verificare l'esistenza con il comando:

```
docker images | grep nodejsapp
```

4) Costruzione dell'immagine del container con mongodb ed il database.

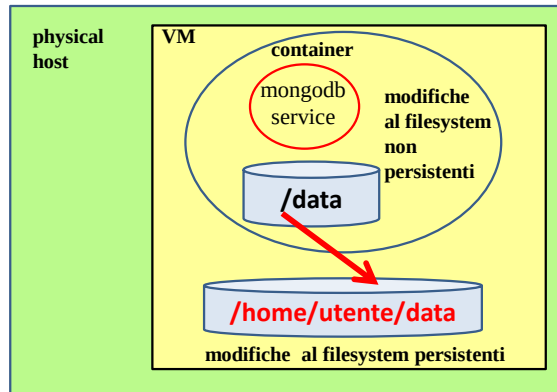
Il tipico container di mongodb inserisce la propria configurazione ed i file in cui memorizza i dati dei suoi database, in una sua directory /data, in particolare in due directory /data/db e /data/configdb

Però, poiché il container mongodb potrebbe essere terminato volutamente o per crash, con la terminazione si perderebbero le modifiche apportate nel frattempo alla base di dati, perché non si salverebbero le modifiche.

Per tale motivo, **il container mongodb è fatto per mappare la propria directory interna /data in una directory esterna al container, cioè una directory della macchina fisica per assicurare la persistenza.**

In questo modo, se anche il container termina, il database rimane nel disco fisico.

Rendere persistenti le modifiche al DB
salvando in volume esterno la
configurazione ed i dati



5) Costruzione dell'immagine del container con mongodb ed il database.

5.2) Scrivere i file necessari ad inizializzare il db mongodb.

Predisponiamo una prima modifica, per inizializzare nel nostro database mongo, un database che chiamo "dbsa" ed una collection che chiamo "alignments" e per inserire almeno un documento nella collection.

Il comando principale del container mongodb è **docker-entrypoint.sh**. Per impostazione predefinita, questo script cerca tutti i file di scripting .js o .sh che, nel container, si trovano nella directory /docker-entrypoint-initdb.d/ e li esegue la prima volta che il container viene messo in esecuzione, o meglio, la volta che il container viene messo in esecuzione e nel db non ci sono database.

Perciò, creo e scrivo un file in linguaggio javascript che farò eseguire dal mio mongodb solo la prima volta che lo metterò in esecuzione. Chiamo il file "mydbinit.js" e questo è il contenuto:

```
var conn = new Mongo();
var db = conn.getDB('dbsa');

// crea collection 'alignments' e la lascia se già esiste
db.createCollection('alignments', function(err, collection) {});

// elimina gli eventuali documenti della collection 'alignments'
// nel caso esistesse già
try { db.alignments.deleteMany( { } ); } catch (e) { print (e); }
```

```
db.alignments.insert({"s1": "GCATGCU", "s2": "GATTACA",  
  "as1": "GCATGC-U", "as2": "G-ATTACA"})
```

Poi assegno i permessi di esecuzione a tutti per gli script:
chmod 777 mydbinit.js

5.3) Guardare il Dockerfile per notare che richiede di usare due volumi esterni (generici) al container, non specificato se named o anonimi.

C'è una riga in cui si crea un volume esterno in cui salvare i dati e la configurazione del db:

```
VOLUME /data/db /data/configdb
```

5.4) Impostare il Dockerfile per far eseguire l'inizializzazione del DB alla prima esecuzione del container.

Aggiungere, subito dopo la riga commentata, una riga in cui si **ordina di copiare il file mydbinit.js** nella directory /docker-entrypoint-initdb.d/ del container in cui mongodb cercherà gli script da eseguire dopo che il container è stato messo in esecuzione.

```
COPY ./mydbinit.js /docker-entrypoint-initdb.d/
```

5.5) Effettuare il build dell'immagine del container.

Abbiamo i file che ci servono, ora li modifichiamo e facciamo il build della nostra immagine.

Creare l'immagine nuova di mongodb, che chiamo "mymongo" lanciando il build che usa il nuovo Dockerfile:

```
docker build -t "mymongo" .
```

NB:C'è un punto alla fine

Se tutto va a buon fine, vedrò tra le immagini del registry locale anche la mia mymongo, lanciando il comando:

```
docker images | grep mymongo
```

Quella è l'immagine del container creato e salvato in locale.

6) Esecuzione del container con mongodb ed il database dbsa

Verifico che la rete virtuale che ho chiamato "interna" e che servirà a connettere i due container sia ancora in esecuzione, eseguendo il comando:

```
docker network ls -f name=interna
```

mi aspetto un output di questo tipo

NETWORK ID	NAME	DRIVER	SCOPE
7e9f77130cd4	interna	bridge	local

Se per caso la rete non esistesse dovrei crearla, come indicato al precedente punto 2, eseguendo il comando:

```
docker network create -d bridge --internal interna
```

Creata la rete virtuale, **metto in esecuzione in background** il container contenente mongodb, partendo dall'immagine che ho creato, eseguendo:

```
docker run -itd --network interna -p 27017-27019:27017-27019  
--name mongodb mymongo
```

Notare che ho assegnato come nome "mongodb" al container che metto in esecuzione.

Notare che ho esposto le porte di mongodb sull'host per potermi collegare PER DEBUGGING DALL'HOST con un applicativo che interroga il database. Normalmente non serve.

Verifico che il container battezzato mongodb sia in esecuzione, invocando il comando:

```
docker ps -a
```

Mi aspetto un output come segue:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
0f70e9a6f83e	mymongo	"docker-entrypoint.s..."	7 minutes ago	Up 7 minutes	
0.0.0.0:27017-27019->27017-27019/tcp		mongodb			

In caso di errori potreste vedere che lo status del container non è Up bensì è **Exited**. In tal caso, lanciate il comando seguente per visualizzare il contenuto del log del container chiamato mongodb e cercare di capire cosa è successo. Potete usarlo in generale per debugging anche mentre il container è ancora in esecuzione.

```
docker logs mongodb
```

Se invece il container è in esecuzione normalmente (status Up), è bene verificare che tutto sia funzionante; perciò faccio eseguire, nel container di mongodb in esecuzione, un comando che usa la shell del client mongo per eseguire uno script .js passato a riga di comando. Questo script interroga mongodb chiedendo di visualizzare il contenuto della collection

```
docker exec -it mongodb /usr/bin/mongo --eval "var conn = new  
Mongo(); var db = conn.getDB('dbsa'); var cursor =
```

```
db.alignments.find(); while ( cursor.hasNext() ) { printjson(
cursor.next() ); }
```

Ci aspettiamo un output come quello che segue, che indica che nel nostro db dbsa c'è un solo documento:

```
MongoDB shell version v4.0.9
connecting to:
mongodb://127.0.0.1:27017/?gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("3df697d6-f2b8-46c5-a69d-43ea51d04b3d") }
MongoDB server version: 4.0.9
{
  "_id" : ObjectId("5cd6a937518b4b8160497e25"),
  "s1" : "GCATGCU",
  "s2" : "GATTACA",
  "as1" : "GCATGC-U",
  "as2" : "G-ATTACA"
}
```

IN CASO DI PROBLEMI CON IL DATABASE, possiamo utilizzare il container mongodb mediante applicativi che si connettono alla porta esposta dal container. ATTENZIONE, riesco a connettermi DALL'HOST PER DEBUGGARE solo se ho esposto la porta 27917. Normalmente dovrei connettermi al database per tramite del web server oppure per tramite di docker exec come sopra spiegato.

Ad esempio, se nell'host locale abbiamo installato il client per connetterci a mongodb (sudo apt install mongodb-clients), allora possiamo connetterci al server mongodb, e in particolare al nostro database dbsa, aprendo come interfaccia la shell di comandi del client mongod, così:

```
mongo 127.0.0.1:27017/dbsa
```

Dall'interno di questa shell interattiva potremo lanciare comandi, ad esempio:

```
use dbsa
db.createCollection("alignments")
db.alignments.find()
db.alignments.insert({"s1": "GCATGCU", "s2": "GATTACA", "as1":
"GCATGC-U", "as2": "G-ATTACA"})
db.alignments.find()
exit
```

Oppure, sempre usando la shell di comandi del client mongo, potremmo direttamente eseguire delle query a riga di comando, passando delle query realizzate con degli script js, ad esempio inserendo dei nuovi documenti, così:

```
mongo 127.0.0.1:27017/dbsa --eval "var conn = new Mongo(); var db
= conn.getDB('dbsa'); db.alignments.insert({'s1\': \"PAZTAZ\",
's2\': \"ZACZAC\", 'as1\': \"PAZTAZ-U\", 'as2\': \"G-
ZACZACVAF\"});"
```

oppure visualizzando quelli esistenti, così:

```
mongo 127.0.0.1:27017/dbsa --eval "var conn = new Mongo(); var db
= conn.getDB('dbsa'); var cursor = db.alignments.find(); while (
cursor.hasNext() ) { printjson( cursor.next() ); }"
```

Il momento in cui decidete di terminare l'esecuzione del vostro container non è critico, poiché le modifiche al database sono salvate sul volume esterno.

Se termino l'applicazione e la faccio ripartire da capo, vedrò i dati del database come erano stati costruiti nella volta precedente.

7) Esecuzione del container con dentro l'applicazione web basata su nodejs, express e mongoose

Ora possiamo fare partire il container con l'applicazione web, collegandolo alla rete interna affinché possa connettersi al database mongodb, usando il comando:

```
docker run -itd --rm --network interna --name nodejsapp -p 3000:3000 nodejsapp
```

Poi colleghiamo il container con il web server alla rete esterna affinché possa essere raggiunta dalle richieste http che arrivano sull'host, mediante il comando:

```
docker network connect esterna nodejsapp
```

Notare che attacco il container alla stessa rete virtuale "**interna**" del container di mongodb. Inoltre assegno nome "**nodejsapp**" al container che metto in esecuzione. Con il parametro **-p 3000:3000** dico che voglio che la porta 3000 del container sia accessibile anche da fuori il container (cioè dall'host), in modo che i client esterni possano accedere al serve nodejs che lavora ed attende sulla porta 3000. Il parametro **--rm** significa che quando il container terminerà l'esecuzione, cioè verrà stoppato, docker dovrà eliminarlo automaticamente.

Per usufruire dell'applicazione, da dentro l'host fisico in cui eseguo il container, posso lanciare un browser grafico o testuale con cui mandare un messaggio HTTP POST con cui passo due parametri seq1 e seq2 e provo così la generazione di 2 stringhe, as1 ed as2, ed il successivo salvataggio in mongodb della quadrupla in formato json {s1,s2,as1,as2} dove s1=seq1 ed s2=seq2.

Ad esempio, lanciando il seguente comando (notare il **/submit**):

```
curl --header "Content-Type: application/x-www-form-urlencoded" --request POST --data 'seq1=ALFABETAGAMMA&seq2=UFFA' 0.0.0.0:3000/submit
```

faccio generare due stringhe as1 ed as2 e viene restituito un documento json che è quello che è stato salvato nel database mongo.

```
{ "s1": "ALFABETAGAMMA", "s2": "UFFA", "as1": "LF-ABETAGAMMA", "as2": "UFFA-----" }
```

Per stoppare il container userò il comando:

```
docker stop <nome container>
```

cioè, nel caso specifico:

```
docker stop nodejsapp
```

Infine, per eliminare il container, dopo averlo stoppato, userò il comando:

```
docker rm <nome container>
```

cioè, nel caso specifico:

```
docker rm nodejsapp
```

Poiché questo container non salva nessun dato, non ha senso fare un commit dopo averlo utilizzato.

8) Automatizzazione di costruzione, dispiegamento ed esecuzione di tutti i container.

Per automatizzare tutto quanto ed eseguirlo con docker compose, è necessaria una modifica del codice dell'applicazione nodejsapp perché per riuscire a connettersi a mongodb occorre aspettare che mongodb sia stato inizializzato.

Inserisco perciò una orrenda pausa di 10 secondi prima di tentare la connessione da nodejsapp a mongodb. Aggiungo inoltre un po' di codice per visualizzare il tipo di errore. Agisco perciò sul file index.js

Tratto dal nuovo file **index.js**

```
// aspetto 10 sec che il container di mongo sia su
function pausecomp(millis)
{
  var date = new Date();    var curDate = null;
  do { curDate = new Date(); } while(curDate-date < millis);
}
pausecomp(10000);

//connessione al db^M
mongoose.set('useFindAndModify', false);
mongoose.set('connectTimeoutMS', 30);
mongoose
  .connect(
    'mongodb://mongodb:27017/dbsa',
    { useNewUrlParser: true })
  .then(() => console.log('MongoDB Connected'))
  .catch((err) => console.log(err));
```

Ora possiamo procedere ad automatizzare creazione ed esecuzione dei container.

Nella directory ASW, radice del mio progetto, **costruisco il file docker-compose.yml** che indica:

- come fare il build delle immagini dei due container,
- come creare la rete virtuale interna e quella esterna,
- come mettere in esecuzione tutti i container.
- come terminare l'esecuzione di tutti i container e rimuoverne le immagini e rimuovere anche le due reti virtuali create

Il file **docker-compose.yml** è fatto come segue:

```
version: '3'
# project name "aswh" è in var COMPOSE_PROJECT_NAME=aswh nel file .env

services:
# il servizio che deve partire per secondo
  nodejsapp:
    build:
      context: ./nodejs
      dockerfile: Dockerfile
    image: nodejsapp
    depends_on:
      mongodb:
        condition: service_healthy
# Questa porta DEVO pubblicarla per vederla dall'host e fuori
dall'host.
    ports:
      - 3000:3000
    networks:
      - interna
      - esterna

# il servizio che deve partire per primo
  mongodb:
    build:
      context: ./mongodb
      dockerfile: Dockerfile
    image: mymongodb
    restart: always
    healthcheck:
      test: |
        host=`hostname --ip-address || echo '127.0.0.1'`;
        mongo --quiet ${host}/test --eval 'quit(db.runCommand({ ping:
1 }) .ok ? 0 : 2)' && echo 0 || echo 1
      interval: 5s
# ENV MONGO_INITDB_DATABASE dbsa
# Questa variabile d'ambiente e' inutile perche' il nome
# del database dbsa viene settato dal codice mydbinit.js
    environment:
      - MONGO_INITDB_DATABASE=dbsa
# Non serve pubblicare le porte anzi non bisogna farlo.
# ports:
```

```

# - 27017-27019:27017-27019
networks:
  - interna
# USO DUE VOLUMI ESTERNI, named, NON ANONIMI, mydb myconfigdb
volumes:
  - type: volume
    source: mydb
    target: /data/db
    volume:
# i files, che al momento della creazione del volume
# sono gia' presenti nella directory del container
# dove deve essere montato il volume,
# non vengono copiati nel volume
      nocopy: true
  - type: volume
    source: myconfigdb
    target: /data/configdb
    volume:
      nocopy: true

networks:
  interna:
    driver: bridge
    internal: true
    driver_opts:
      com.docker.network.bridge.name: "br-interna00000"
  esterna:
    driver: bridge
    internal: false
    driver_opts:
      com.docker.network.bridge.name: "br-esterna000000"

volumes:
  mydb:
  myconfigdb:

```

Per costruire le immagini dei container, stando nella directory del file docker-compose.yml, eseguire il comando:

docker-compose build

Per mettere in esecuzione tutti i container, creando le reti necessarie, stando nella directory del file docker-compose.yml, eseguire il comando:

docker-compose up -d

Per terminare l'esecuzione di tutti i container, eseguire il comando:

docker-compose stop

Per terminare l'esecuzione di tutti i container, ed anche eliminare i container e le reti create, eseguire il comando:

docker-compose down

Per terminare l'esecuzione di tutti i container, ed anche eliminare i container e le reti create, **ed eliminare pure le immagini dei container creati**, eseguire il comando:

docker-compose down --rmi all

Se oltre ad eliminare container, reti e immagini voglio eliminare anche i volumi creati automaticamente per la mia applicazione, allora devo aggiungere l'opzione **-v**, cioè devo eseguire il comando:

docker-compose down --rmi all -v

Notare che docker-compose assegna ai container e alle reti che crea dei nomi strani ma prevedibili. Infatti, se:

- la directory in cui si trova il file docker-compose.yml si chiama **Aswh4**,
- un servizio da creare nel file si chiama **mongodb**,
- di quel servizio viene creata una sola replica,
- la rete l'ho chiamata **"interna"**,

allora docker-compose fa sì che:

- il container generato per quel servizio si chiamerà **aswh4-mongodb-1**,
- la rete si chiamerà **aswh_interna**,
- l'host virtuale in cui è contenuto il container si chiamerà come il servizio **mongodb**, ma anche come il container, **aswh-mongodb-1**, o anche **aswh-mongodb-1.aswh_interna**.

Nota Bene: però posso decidere io il nome del progetto (aswh4) che costituisce il prefisso delle risorse create. Basta creare nella directory base del progetto, cioè quella che contiene il file docker-compose.yml, **un file che si chiama .env** e che contiene una riga che setta una variabile così (deve avere caratteri minuscoli):

COMPOSE_PROJECT_NAME=aswh4